UNITÀ 7 · IL SISTEMA OPERATIVO COME GESTORE DI RISORSE

L'allocazione contigua a partizioni

Tanti processi, una sola RAM: dove li mettiamo? Tre strategie, una sfida a squadre e un nemico invisibile, la frammentazione.

PER COMINCIARE

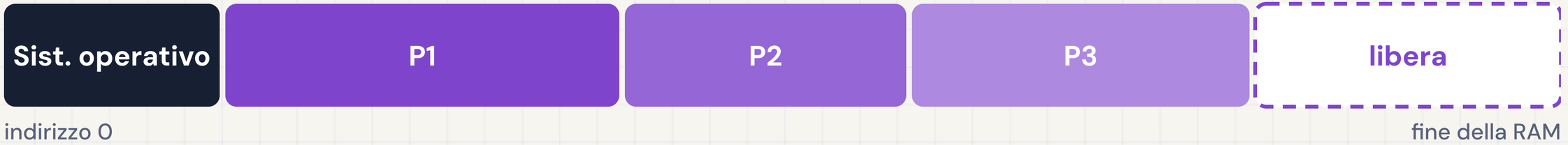
Arrivi in un parcheggio con posti di lunghezze diverse. Dove ti metti?

SONDAGGIO

- **A** Nel primo posto libero in cui entro
- **B** Nel posto più stretto in cui entro
- **C** Nel posto più largo di tutti
- **D** Giro finché non trovo quello perfetto

IL PROBLEMA

Allocazione contigua: ogni processo in un blocco unico



- Ogni processo occupa indirizzi **consecutivi**
- Bastano un registro base e un registro limite
- Il sistema operativo tiene la lista delle zone libere
- Domanda: come divido la RAM?

PRIMA SOLUZIONE

Partizioni fisse: la RAM tagliata in anticipo

All'avvio la RAM viene divisa in partizioni di dimensioni **fisse**. Ogni processo va in una partizione abbastanza grande.

Partizione	Dimensione	Processo	Sprecato
1	100 KB	libera	-
2	200 KB	P1, 130 KB	70 KB
3	400 KB	P2, 380 KB	20 KB

Frammentazione interna

Lo spazio sprecato **dentro** una partizione assegnata. Nessun altro processo può usarlo.

E un processo di 450 KB? Non potrà mai partire.

METTITI ALLA PROVA

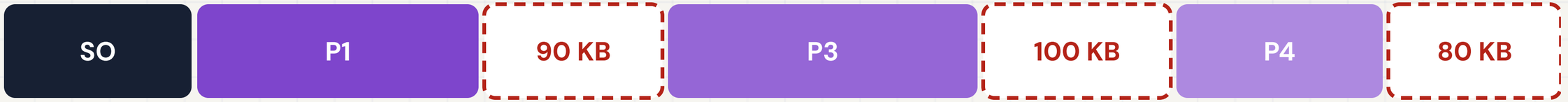
Partizioni fisse da 100, 200 e 300 KB. Arrivano tre processi da 90, 160 e 250 KB. Quanto si spreca?

QUIZ

- **A** 0 KB: entrano tutti
- **B** 50 KB
- **C** 500 KB
- **D** 100 KB

SECONDA SOLUZIONE

Partizioni variabili: ogni processo riceve quanto chiede



Frammentazione esterna

Liberi in totale 270 KB, ma in tre **buchi** separati: un processo da 200 KB non entra.

MENTI CHE CI HANNO PRECEDUTO

OS/360: partizioni fisse e variabili, e una lezione sui progetti

> **Aggiungere persone a un progetto software in ritardo lo fa ritardare ancora di più.**
>
> **Fred Brooks**, responsabile dello sviluppo di OS/360 per IBM, in "The Mythical Man-Month", 1975

Il sistema operativo OS/360 (anni '60) offriva due versioni: **MFT**, con un numero fisso di partizioni, e **MVT**, con partizioni variabili. Proprio quello che studiamo oggi.

LE STRATEGIE

Arriva un processo: in quale buco lo metto?

First fit

Il primo che va bene

Scorri i buchi dall'inizio e prendi il primo abbastanza grande.

Veloce: si ferma subito.

Best fit

Il più piccolo che va bene

Guarda tutti i buchi e prendi quello che lascia l'avanzo minimo.

Lascia briciole minuscole.

Worst fit

Il più grande di tutti

Prendi il buco più grande, così l'avanzo resta utilizzabile.

Consuma i buchi grandi.

Regola per gli esercizi: l'avanzo resta nella stessa posizione e diventa un buco più piccolo.

ESEMPIO SVOLTO

Buchi A 150, B 400, C 220, D 90, E 350 (KB)

Richiesta	First fit	Best fit
P1 · 200 KB	B, restano 200	C, restano 20
P2 · 80 KB	A, restano 70	D, restano 10
P3 · 300 KB	E, restano 50	E, restano 50
P4 · 140 KB	B, restano 60	A, restano 10
Buchi finali	`70 · 60 · 220 · 90 · 50`	`10 · 400 · 20 · 10 · 50`

Stesso spazio libero (490 KB) in entrambi i casi, ma best fit conserva un buco da 400 KB.

TOCCA A TE

Stessi buchi, stesse richieste, ma con worst fit. Chi resta fuori?

QUIZ

- **A** Nessuno, entrano tutti
- **B** P4 da 140 KB
- **C** P3 da 300 KB
- **D** P1 da 200 KB

AL COMPUTER · 15 minuti

Calc sceglie il buco per te

1. In A2:A6 scrivi 1…5 (numero del buco), in B2:B6 i KB: 150, 400, 220, 90, 350. In G1 la richiesta: 200.
2. In C2 `=SE(B2>=$G$1;B2;"")`, in D2 `=SE(B2>=$G$1;A2;"")`; copia fino alla riga 6.
3. First fit: in G3 `=MIN(D2:D6)`. Best fit: in G4 `=MIN(C2:C6)`. Worst fit: in G5 `=MAX(C2:C6)`.
4. Prova anche le richieste 80 e 500. Salva come **scegli-buco.ods**.

Controllo: con 200 leggi 2, 220, 400. Con 80: 1, 90, 400. Con 500 leggi 0: nessun buco va bene, il processo aspetta.

SFIDA A SQUADRE

Il torneo degli algoritmi

Buchi: **A 200 · B 450 · C 100 · D 300** (KB)

Richieste, in ordine: **90 · 180 · 280 · 120 · 240**

1. Ogni squadra pesca un algoritmo: first, best o worst fit.
2. Sul foglio aggiorna la lista dei buchi dopo ogni richiesta.
3. Vince l'algoritmo che sistema più processi; a parità, chi lascia il buco più grande.

⏱ 6:00

> Attenzione: ogni squadra deve poter spiegare ogni scelta alla classe.

QUIZ A TRABOCCHETTO

Allora best fit è sempre il migliore?

QUIZ

A Sì, perché spreca sempre meno

B No, dipende dalla sequenza di richieste

C Sì, ma solo con richieste piccole

D No, worst fit è sempre il migliore

IL RIMEDIO

Compattazione: spingere tutti i processi da una parte

Il sistema operativo **sposta** i processi uno accanto all'altro: i buchi si uniscono in un unico grande buco.

- Si può fare perché basta cambiare il registro **base**
- Costa molto: bisogna copiare tanta memoria
- Durante la copia i processi sono fermi



AL COMPUTER · 15 minuti

Disegna la RAM e compattala

1. In Calc, celle A1:A20: ogni cella vale **50 KB** (in tutto 1000 KB).
2. Colora: SO 2 celle (nero), P1 4, libere 2, P2 3, libere 3, P3 3, libere 3. Scrivi il nome in ogni blocco.
3. In C1 annota quanti buchi ci sono e quanto è grande il più grande. Entra un processo da 300 KB?
4. In colonna E ridisegna la RAM dopo la compattazione e rispondi di nuovo.
5. Salva come **mappa-memoria.ods**.

Controllo: prima: 3 buchi (100, 150, 150 KB), 400 KB liberi, il processo da 300 non entra. Dopo: un solo buco da 400 KB in fondo, e il processo entra.

SIMULAZIONE IN JAVA

First fit in 15 righe

```
public class Allocazione {
  public static void main(String[] args) {
    int[] buchi = {150, 400, 220, 90, 350};
    int[] richieste = {200, 80, 300, 140};
    for (int r : richieste) {
      int scelto = -1;
      for (int i = 0; i < buchi.length; i++) {
        if (buchi[i] >= r) { scelto = i; break; }   // first fit
      }
      if (scelto == -1) { System.out.println(r + " KB: aspetta"); continue; }
      buchi[scelto] -= r;
      System.out.println(r + " KB -> " + (char) ('A' + scelto) + ", restano " + buchi[scelto]);
    }
  }
}
```

AL COMPUTER · 20 minuti

Da first fit a best fit cambiando una riga

1. Su **onlinegdb.com** scegli **Java**, copia il programma della slide precedente e premi **Run**.
2. Sostituisci la riga evidenziata con questa ed esegui di nuovo: ora è best fit.

```
if (buchi[i] >= r && (scelto == -1 || buchi[i] < buchi[scelto])) scelto = i;
```

3. Cambia `<` in `>` ed esegui. Salva come **Allocazione.java** (Download).

Controllo: first fit: B 200, A 70, E 50, B 60. Best fit: C 20, D 10, E 50, A 10. Con `>` è worst fit: B 200, E 270, "300 KB: aspetta", E 130. Uguale alla tabella!

PUZZLE

Abbina ogni termine alla sua descrizione

Frammentazione interna	Spazio libero sufficiente, ma spezzato in buchi
Frammentazione esterna	Il primo buco abbastanza grande
Compattazione	Il buco che lascia l'avanzo più piccolo
First fit	Spazio sprecato dentro una partizione assegnata
Best fit	Il buco più grande disponibile
Worst fit	Spostare i processi per unire i buchi

SFIDA FINALE

Apri il lucchetto con l'esempio svolto



- 1ª cifra: con worst fit, il numero del processo che deve aspettare
- 2ª cifra: con best fit, la prima cifra del buco più grande rimasto
- 3ª cifra: quante strategie "fit" abbiamo confrontato
- 4ª cifra: lo spreco del quiz sulle partizioni fisse, diviso 100

codice

BIGLIETTO D'USCITA

Completa prima di uscire

Lo spazio sprecato dentro una partizione si chiama frammentazione _____.

Buchi liberi sparsi e troppo piccoli: frammentazione _____.

La strategia che prende il buco più piccolo adatto è il _____.

Con buchi 120, 300, 200 e richiesta 150, first fit sceglie il buco da _____ KB.

Unire i buchi spostando i processi si chiama _____.

⏱ 3:00

RIEPILOGO

Cosa portiamo a casa

- Contigua: ogni processo in un blocco unico
- Partizioni fisse → frammentazione interna
- Partizioni variabili → frammentazione esterna
- First, best, worst fit: nessuno vince sempre
- Compattazione: funziona, ma costa
- La vera soluzione? Le pagine, la prossima volta

Hai una domanda su una slide? Usa il pulsante **? Domande** in basso: la vedrà il docente e tutta la classe.